# L3-4 讲义：Workflow 与 RAG

**课程**: MBA 大模型智能体课程
**课时**: 第 3-4 讲 (共 3 小时)
**适用**: 讲师备课 / 学生预习与复习

---

## 本讲关键问题

作为 MBA 学生，学完这一讲你应该能回答：

1. **为什么不能直接用 ChatGPT 完成所有任务？** → 单次调用的局限
2. **如何把复杂任务变成 AI 可以执行的步骤？** → Workflow 设计
3. **如何让 AI 使用我们公司的私有数据？** → RAG 原理与实践
4. **RAG 和微调哪个适合我们？** → 技术选型决策
5. **如何评估 RAG 系统的效果？** → 评估指标

---

## 为什么这节课很重要

上节课我们学了 LLM 的能力和局限。你可能会想："既然 ChatGPT 这么强，为什么还要学别的？"

**现实情况**：直接用 ChatGPT 在企业场景有几个致命问题：

| 问题 | 具体表现 | 商业影响 |
|---|---|---|
| **知识过时** | "2024 年特斯拉股价？"→ 不知道 | 无法做实时分析 |
| **没有私有数据** | "我们 Q3 销售额？"→ 不知道 | 无法处理内部信息 |
| **会编造事实** | "竞争对手最新动态？"→ 可能胡说 | 误导决策 |
| **不能执行操作** | "帮我发一封邮件"→ 只能写，不能发 | 无法自动化 |

**这节课的价值**：学会用 Workflow 和 RAG 解决这些问题，让 AI 真正为你的业务服务。

---

## 目录


1. 从 LLM 到 Workflow
2. Workflow 设计模式
3. RAG 基础
4. RAG 进阶优化
5. 实践案例
6. 参考资料


---

## 1. 从 LLM 到 Workflow

### 1.1 单次 LLM 调用的局限—一个真实案例

**场景**：你让 AI 分析一份 100 页的行业报告并写出要点。

**问题 1：上下文窗口限制**

即使是最新的模型（如 Claude 200K），处理超长文档也有问题：- 100 页报告约 15 万字 ≈ 20 万 tokens - 可能超出窗口，或者"中间迷失"（模型容易忘记中间内容）

**解决方案**：分段处理 → 汇总

**问题 2：没有最新信息**

你问："这份报告中的市场数据和最新情况对比如何？"- LLM 的知识有截止日期 - 它不知道今天发生了什么

**解决方案**：先搜索最新数据 → 再让 LLM 对比分析

**问题 3：只能说，不能做**

你说："分析完之后把要点发给团队"- LLM 只能写出邮件内容 - 不能真的发送

**解决方案**：LLM 生成内容 → 调用邮件 API 发送

**这就是 Workflow 的价值**：把"LLM 不能直接做的事"拆成"LLM + 工具"可以做的步骤。

### 1.2 什么是 Workflow?

**简单定义**：Workflow = 编排多个步骤（LLM 调用 + 工具调用 + 逻辑控制）

**商业类比**：想象一条生产线

```
原材料 → 工序1 → 工序2 → 工序3 → 成品
            ↓       ↓       ↓
           检查    加工    包装
```

AI Workflow 是一样的逻辑：

```
用户需求 → 步骤1(理解) → 步骤2(检索) → 步骤3(分析) → 步骤4(生成) → 交付结果
            ↓            ↓            ↓            ↓
           LLM         数据库        LLM          LLM
```

**关键区别**：- **人工处理**：每一步都要人盯着 - **Workflow**：设计好流程后，自动执行

### 1.3 Workflow vs Agent —MBA 最常问的问题

"我听说 Agent 很火，为什么还要学 Workflow？"

**简短回答**：Workflow 是 Agent 的基础，而且很多场景 Workflow 就够了。

**详细对比**：

| 维度 | Workflow | Agent |
|---|---|---|
| **执行路径** | 你预先设计好的 | AI 自己决定 |
| **可预测性** | 高（每次走一样的路） | 低（每次可能不同） |
| **成本** | 可控（步骤固定） | 不可控（可能循环很多次） |
| **适用场景** | 流程明确、有标准答案 | 开放式、需要探索 |
| **类比** | 按菜谱做菜 | 看冰箱有什么再决定做什么 |

**决策原则**：

```
能用Workflow解决的 → 优先用Workflow
                        ↓
            因为：更便宜、更可控、更好调试
                        ↓
            只有确实需要"自主决策"时 → 才用Agent
```

**真实案例对比**：

| 任务 | 推荐 | 原因 |
|---|---|---|
| 每日新闻摘要 | Workflow | 步骤固定：抓取 → 筛选 → 摘要 → 发送 |
| 客户投诉处理 | Agent | 每个投诉不同，需要判断和决策 |
| 财务报表生成 | Workflow | 数据来源和格式都固定 |
| 竞品深度分析 | Agent | 需要搜索、判断、深挖 |

---

## 2. Workflow 设计模式

就像建筑有不同的结构（平房、高楼、桥梁），Workflow 也有不同的设计模式。选对模式事半功倍。

### 2.1 模式 1：顺序链 (Sequential Chain)

**最简单的模式**：步骤一个接一个，像流水线。

输入 → Step1 → Step2 → Step3 → 输出

**适用场景**：有明确先后顺序的任务

**真实案例：多语言内容发布**

```
原始文章(中文)
    ↓
Step1: 翻译成英文 (LLM)
    ↓
Step2: 文化适配调整 (LLM)
    ↓
Step3: SEO优化 (LLM)
    ↓
Step4: 格式转换 (代码工具)
    ↓
发布到海外平台
```

**关键设计点**：- 每一步的输出是下一步的输入 - 如果中间一步失败，整个流程停止 - 易于调试：可以检查每一步的中间结果

### 2.2 模式 2：并行扇出 (Fan-out)

**多个任务同时执行**，最后汇总结果。

```
          → 任务A
输入    → 任务B   → 汇总 → 输出
          → 任务C
```

**适用场景**：多个独立子任务可以并行处理

**真实案例：投资尽调报告**

```
目标公司信息

          → 财务分析 (分析师A/LLM)

          → 法律风险 (法务/LLM)        →→→ 汇总成
                                           尽调报告
          → 技术评估 (技术专家/LLM)

          → 市场分析 (市场部/LLM)
```

**商业价值**：- 传统方式：串行执行需要 4 天 - 并行 Workflow：所有分析同时进行，1 天完成 - **效率提升 4 倍**

### 2.3 模式 3：条件分支 (Routing)

**根据输入决定走哪条路**，像高速公路的分叉口。

```
输入 → 分类器  → 路径A → 处理A
               → 路径B → 处理B
               → 路径C → 处理C
```

**适用场景**：不同类型的输入需要不同处理

**真实案例：智能客服系统**

```
用户消息
    ↓
意图识别(LLM)

      "退款" → 退款处理流程 → 查订单 → 判断是否符合 → 执行/拒绝

      "咨询" → FAQ检索 → 返回答案

      "投诉" → 创建工单 → 转人工

      "其他" → 通用对话(LLM)
```

**关键设计点**：- 分类器的准确率决定整个系统的效果 - 要覆盖所有可能的情况（包括"其他"兜底）- 每条路径可以是不同复杂度的子 Workflow └→ 投诉 → 人工升级

### 2.4 循环迭代（Loop）

**任务需要反复改进**，直到满足条件。

输入 → 处理 → 检查 ─┬→ 通过 → 输出 └→ 不通过 → 修改 → 处理 (循环)

**适用场景**：需要迭代优化的任务

**真实案例：AI辅助写作**

写作需求:"写一篇关于 AI 在零售业应用的文章"↓ Step1: LLM 生成初稿 ↓ Step2: 检查质量 (另一个 LLM 或规则) │ ├── 通过 → 输出文章 │ └── 不通过 → 识别问题 → 针对性修改 → 回到 Step2 │ └──"论据不足" →"增加数据支撑" └──"结构混乱"→"重新组织段落"

**风险控制**:
- 必须设置最大循环次数（否则可能无限循环）
- 每次循环都消耗API成本
- 建议：3-5次循环后强制输出

### 2.5 人工介入（Human-in-the-Loop）

**关键节点需要人工审核**，这是生产系统中最重要的模式之一。

输入 → AI 处理 → 人工审核 ─┬→ 批准 → 执行 └→ 拒绝 → 终止/修改

**为什么需要人工介入？**

| 场景 | 为什么需要人工 |
|---|---|
| 合同审批 | 法律风险需要专业判断 |
| 大额退款 | 财务影响需要人工确认 |
| 对外发布 | 品牌声誉风险 |
| 医疗建议 | 生命安全 |

**设计要点**:
- 明确哪些情况触发人工审核
- 给人工审核者足够的上下文
- 记录审核决策，用于后续模型优化

---

## 3. RAG基础 — 让AI使用你的私有数据

### 3.1 一个商业场景理解RAG

**场景**: 你是一家咨询公司，有10年积累的行业报告（共500份）。新员工入职后问: "我们对零售行业的观点是什么

**问题**:
- 直接问ChatGPT → 它不知道你们公司的报告
- 让新员工翻500份报告 → 太慢，可能漏掉关键信息
- 把500份报告全部粘贴给AI → 超出context window

**RAG的解决方案**:

1. 预处理（一次性）: 500 份报告 → 切成小块 → 转成向量 → 存入数据库
2. 查询（每次）: “零售行业观点”→ 搜索相关段落 → 找到最相关的 10 段 ↓ → LLM 基于这 10 段生成回答

**效果**: 新员工问任何问题，AI都能从500份报告中找到相关内容并回答。

### 3.2 RAG是什么?

**RAG = Retrieval-Augmented Generation（检索增强生成）**

**一句话解释**: 先搜索相关资料，再让AI基于资料回答。

**对比**:

| 方式 | 过程 | 优点 | 缺点 |
|---|---|---|---|
| **纯LLM** | 直接问AI | 简单 | 可能编造、知识过时 |
| **RAG** | 先检索→再生成 | 有据可依、可更新 | 需要搭建检索系统 |
| **微调** | 用数据重新训练 | 深度融合知识 | 成本高、不易更新 |

**RAG的商业优势**:
- **低成本**: 不需要训练模型，只需要搭建检索系统
- **易更新**: 有新文档? 加入数据库即可
- **可追溯**: 可以显示"这个回答来自哪份文档"
- **保护隐私**: 私有数据不用发给模型提供商训练

### 3.3 RAG工作原理

**两个阶段**:

**阶段一：离线索引（准备工作，一次性）**

你的文档（PDF/Word/网页…）↓ 切分成小块 (每块 500-1000 字) ↓ 转成向量 (用 Embedding 模型) ↓ 存入向量数据库

**阶段二：在线查询（每次用户提问）**

用户问题:“我们对零售行业的观点?”↓ 问题转成向量 ↓ 在向量数据库中搜索 (找语义最相似的文档块) ↓ 取出 Top-K 个相关块 (比如最相关的 5 个) ↓ 拼接成 Prompt:“根据以下资料回答问题：[资料 1]…[资料 2]…问题：我们对零售行业的观点?”↓ LLM 生成回答

### 3.4 核心概念详解

#### (1) 文本切分（Chunking）— 为什么要切？切多大？

**为什么要切分？**
- 原始文档可能很长（100页PDF）
- Context window有限，不能全塞进去
- 检索需要精确到段落级别，不是整个文档

**切多大合适？**

| Chunk大小 | 优点 | 缺点 | 适用场景 |
|---|---|---|---|
| **小块(200字)** | 检索精准 | 可能丢失上下文 | FAQ、定义类 |
| **中块(500-1000字)** | 平衡 | - | 通用场景 |
| **大块(2000字+)** | 上下文完整 | 检索不精准 | 长文档分析 |

**最佳实践**:
- 起步建议：500-1000字
- 加上重叠（overlap）：每块和下一块重叠100-200字，避免切断句子

#### (2) 向量嵌入（Embedding）— 文字怎么变成数字？

**商业类比**：想象把每段文字放到一个"语义地图"上

- "苹果公司财报"和"Apple earnings report" → 地图上很近
- "苹果公司财报"和"今天天气很好" → 地图上很远

**技术原理（简化）**:
- Embedding模型把文本转成一串数字（向量）
- 语义相似的文本，向量也相似
- 搜索时比较向量之间的"距离"

**常用Embedding模型**:

| 模型 | 来源 | 特点 |
|---|---|---|
| text-embedding-3-small | OpenAI | 便宜、通用 |
| text-embedding-3-large | OpenAI | 效果更好、更贵 |
| BGE-large-zh | 智源 | 中文效果好、开源 |

#### (3) 向量数据库 — 专门存储和搜索向量

**为什么需要专门的数据库？**
- 普通数据库（MySQL）不擅长向量相似度搜索
- 向量数据库专门优化了这种搜索，可以在百万级数据中毫秒级返回结果

**常用选项**:

| 数据库 | 特点 | 适用场景 |
|---|---|---|
| **Pinecone** | 云服务、免运维 | 快速上线、企业级 |
| **Weaviate** | 开源、功能丰富 | 需要定制化 |
| **Chroma** | 轻量级、易上手 | 原型开发 |
| **Milvus** | 高性能、开源 | 大规模部署 |

### 3.5 RAG vs 微调 — 如何选择？

**这是MBA最关心的决策问题**: 我们应该用RAG还是微调模型？

| 维度 | RAG | 微调（Fine-tuning） |
|---|---|---|
| **成本** | 低（搭建检索系统） | 高（需要训练） |
| **知识更新** | 容易（添加文档） | 困难（需要重新训练） |
| **可追溯** | 是（可以显示来源） | 否（融入模型权重） |
| **适用数据量** | 任意 | 需要大量数据 |
| **效果** | 依赖检索质量 | 知识融合更深 |
| **适用场景** | 知识库问答、文档检索 | 特定风格/领域适配 |

**决策框架**:

你的需求是什么？

知识库问答（公司文档、FAQ）→ RAG ↓ 知识会频繁更新吗？→ 是 → RAG ↓ 需要显示信息来源吗？→ 是 → RAG ↓ 需要模型学会特定写作风格？→ 是 → 微调 ↓ 有大量高质量训练数据？→ 是 → 微调 ↓ 预算充足？→ 是 → 可以 RAG+ 微调结合

**实践建议**: 80%的企业场景，RAG就够了。

### 3.6 基础实现（给技术同事看）

```python
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA

# 1. 加载文档
loader = PyPDFLoader("company_report.pdf")
documents = loader.load()

# 2. 切分文档
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```

```
chunks = splitter.split_documents(documents)

# 3. 创建向量存储
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)

# 4. 创建RAG链
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4"),
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5})
)

# 5. 查询
answer = qa_chain.run("公司2024年的营收是多少？")
```

---

## 4. RAG 进阶优化

### 4.1 Chunking 策略

| 策略 | 说明 | 适用场景 |
|---|---|---|
| **固定长度** | 按字符数切分 | 简单场景 |
| **递归切分** | 按段落 → 句子 → 字符层级切分 | 通用场景 |
| **语义切分** | 按语义边界切分 | 要求高精度 |
| **文档结构** | 按标题/章节切分 | 结构化文档 |

**最佳实践**：- Chunk 大小：500-1500 字符 - Overlap：10-20% - 保留元数据（来源、页码）

### 4.2 Contextual Retrieval

- ☐ 来源: Anthropic - Contextual Retrieval

**问题**：切分后的 chunk 丢失上下文

**解决方案**：为每个 chunk 添加上下文说明

```
原始chunk：
"第三季度收入增长了15%"

添加上下文后：
"[来自2024年Q3财报，第三章：业绩概览] 第三季度收入增长了15%"
```

**效果**：- 检索失败率从 5.7% 降至 1.9% - 结合 reranking 可降至 1.5%

### 4.3 混合检索 (Hybrid Search)

```
用户查询 → 向量检索（语义相似） → 融合排序 → Top-K结果
         → 关键词检索（BM25）
```

**优势**：- 向量检索：理解语义，处理同义词 - 关键词检索：精确匹配，处理专业术语 - 混合：两者优势互补

### 4.4 Reranking

```
初步检索 → Top-20文档 → Reranker模型 → 重排序 → Top-5文档
```

**常用 Reranker**：- Cohere Rerank - BGE Reranker - Cross-encoder 模型

### 4.5 Query Transformation

| 技术 | 说明 | 示例 |
|---|---|---|
| Query Expansion | 扩展查询词 | "AI"→ "AI, 人工智能, machine learning" |
| HyDE | 先生成假设答案再检索 | 问题 → 假设答案 → 用答案检索 |
| Multi-Query | 生成多个查询变体 | 一个问题生成 3 种问法 |
| Step-back | 先问更高层次问题 | "苹果股价"→ 先了解"科技股大盘" |

### 4.6 RAG 评估指标

| 指标 | 说明 | 计算方式 |
|---|---|---|
| Recall@K | 相关文档被检索到的比例 | 检索到的相关文档/总相关文档 |
| Precision@K | 检索结果中相关的比例 | 相关文档/检索到的文档 |
| MRR | 第一个相关结果的排名 | 1/第一个相关结果的位置 |
| Answer Relevance | 回答与问题的相关性 | LLM 评分 |
| Faithfulness | 回答是否基于检索文档 | LLM 评分 |

---

## 5. 实践案例

### 5.1 企业知识库问答

**场景**：员工查询公司政策、流程

**架构**：

```
员工问题
    ↓
意图识别（LLM）
        政策查询 → RAG(政策文档库)
        流程查询 → RAG(流程文档库)
        其他 → 通用LLM
    ↓
答案 + 来源引用
```

**优化点**：- 多库检索，按领域分类 - 强制引用来源，减少幻觉 - 设置置信度阈值，低于阈值转人工

### 5.2 财报分析 Workflow

**场景**：自动分析上市公司财报

**流程**：

```
财报PDF
    ↓
Step1：解析提取（表格识别、OCR）
    ↓
Step2：数据结构化（LLM提取关键指标）
    ↓
Step3：同行对比（RAG检索同行数据）
    ↓
Step4：趋势分析（计算同比环比）
    ↓
```

```
Step5: 报告生成（LLM生成分析报告）
    ↓
Step6: 人工审核（分析师review）
    ↓
最终报告
```

### 5.3 智能客服

**场景**：处理客户咨询和投诉

**Workflow**：

```
客户消息
    ↓
Step1: 意图分类
       咨询 → RAG(FAQ库) → 直接回答
       投诉 → 情感分析 → 安抚 + 转人工
       退款 → 规则检查 → 自动/人工处理
    ↓
满意度调查
```

---

## 6. 参考资料

### 核心阅读

- Anthropic: Contextual Retrieval - RAG 优化
- LangChain RAG Tutorial

### 工具文档

- LangChain
- LlamaIndex
- Chroma
- Pinecone

### 论文

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- Lost in the Middle - 长上下文检索问题

---

## 课后思考题

1. 设计一个你业务场景的 Workflow，画出流程图
2. 为你的知识库设计 chunking 策略，说明理由
3. 如何评估 RAG 系统的效果？设计 3 个评估指标

---

本讲义最后更新：2026-02-27